

Projektdokumentation

295 - Backend für Applikationen realisieren

Florina Osmani

Datum: 11.07.2026

Version: 1.0

Inhaltsverzeichnis

1. Projektidee.....	1
2. Anforderungskatalog	2
3. Klassendiagramm.....	4
4. Ablauf der User Stories	5
4.1 US1 – Alle Wörter abrufen	5
4.1.1 Ablauf	5
4.1.2 Datentypen je Endpoint	5
4.2 US2 – Definition favorisieren	5
4.2.1 Ablauf	5
4.2.2 Datentypen je Endpoint	6
4.3 US3 – Neues Wort mit mehreren Definitionen anlegen	6
4.3.1 Ablauf	6
4.3.2 Datentypen je Endpoint	7
4.4 US4 – Wort löschen	7
4.4.1 Ablauf	7
4.4.2 Datentypen je Endpoint	8
5. Testplan.....	9
6. Installationsanleitung	11
6.1 Voraussetzungen	11
6.2 Schritte	11
7. Hilfestellungen.....	12

1. Projektidee

Dieses Projekt bietet das Backend zur Wörterbuch-Applikation WordBank vom vorherigen Modul 294. Es ermöglicht Benutzer*innen englische Vokabeln zu entdecken, zu favorisieren und mit persönlichen Notizen zu versehen.

Das Backend wird als RESTful API mit Java Spring Boot realisiert, und stellt alle notwendigen Endpunkte für das Frontend bereit. Es verwaltet Wörter mit ihren zugehörigen Definitionen, Synonymen, Antonymen und Beispielsätzen in einer relationalen Datenbank. Dabei wird die Beziehung zwischen einem Wort und seinen mehreren Definitionen korrekt abgebildet und persistiert.

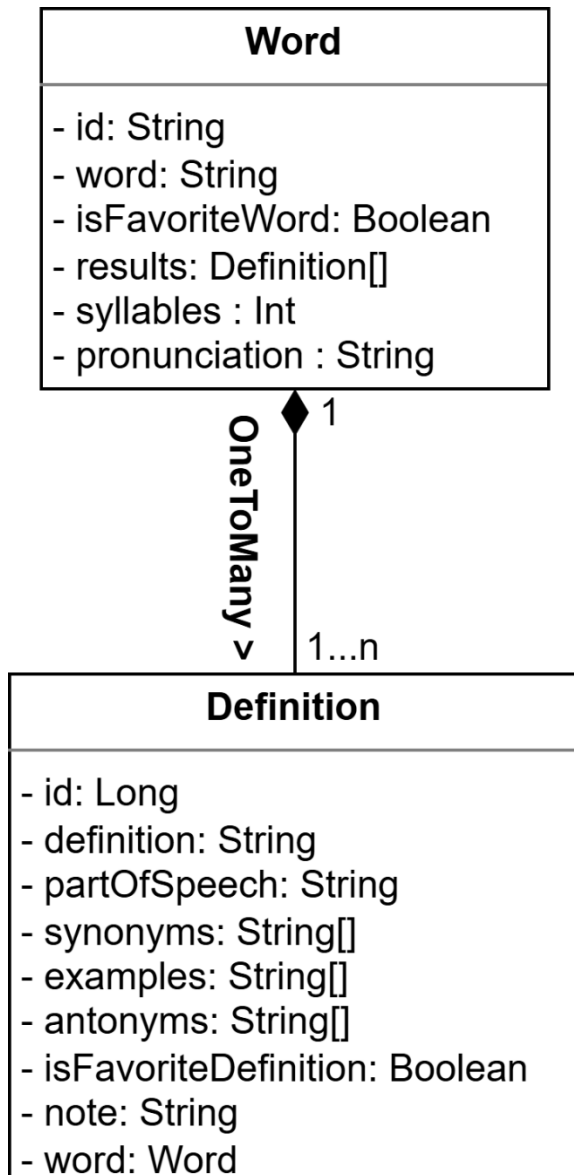
Die API ermöglicht es Benutzer*innen, zufällige Wörter abzurufen, Definitionen als Favoriten zu markieren und mit individuellen Notizen zu ergänzen. Administratoren können zusätzlich neue Wörter mit beliebig vielen Definitionen erfassen oder bestehende Einträge löschen, um die Wortbank aktuell zu halten. Das Backend stellt dabei bewusst einen vollständigen Update-Endpunkt bereit, auch wenn das Frontend diesen nur für die Favorisierung und Notizverwaltung nutzt. So bleibt die API flexibel und unabhängig vom Frontend erweiterbar.

2. Anforderungskatalog

ID	Name	User Story	Akzeptanzkriterien
US1	Alle Wörter abrufen	Als Benutzer*in möchte ich alle verfügbaren Wörter vom Server erhalten, damit mir das Frontend daraus ein zufälliges anzeigen kann.	• GET-Endpoint liefert alle Word-Objekte inkl. ihrer Definitionen. • Bei leerer Datenbank wird ein leeres Array zurückgegeben (kein Fehler). • Antwort enthält pro Wort Aussprache, Silbenzahl und alle Definitionen.
US2	Definition favorisieren*	Als Benutzer*in möchte ich eine Definition als Favorit markieren, damit ich sie später wiederfinde.	• PUT-Endpoint aktualisiert "isFavoriteDefinition" und "isFavoriteWord" auf true. • Bei ungültiger ID wird 404 Not Found zurückgegeben. • Response enthält aktualisiertes Word-Objekt.
US3	Neues Wort mit mehreren Definitionen anlegen	Als Admin möchte ich ein neues Wort mit mehreren Definitionen erfassen, damit die Wortbank wächst.	• POST-Endpoint validiert Pflichtfelder serverseitig. • Bei fehlenden Pflichtfeldern: 400 Bad Request mit Fehlerdetails • Erfolgreiche Anfrage gibt 201 Created mit generierter ID zurück
US4	Wort löschen	Als Admin möchte ich ein Wort löschen können, damit veraltete Einträge entfernt werden	• DELETE-Endpoint entfernt Wort inkl. aller Definitionen • Bei nicht existierender ID wird 404 Not Found zurückgegeben. • Bei erfolgreichem Löschen wird 204 No Content.

*Der PUT-Endpunkt wird als vollständiger Update-Endpunkt implementiert und akzeptiert das gesamte Word-Objekt. Das Frontend nutzt ihn jedoch bewusst nur für die Favorisierung und Notizverwaltung.

3. Klassendiagram



4. Ablauf der User Stories

4.1 US1 – Alle Wörter abrufen

4.1.1 Ablauf

1. Frontend startet → GET /api/words
2. Frontend wählt zufälliges Wort aus der Response aus und zeigt es an

4.1.2 Datentypen je Endpoint

GET /api/words

Request-Body: Kein Body

Response-Status: 200 OK

Response-Body:

```
[
  {
    "id": "4",
    "word": "diligent",
    "results": [
      {
        "definition": "having or showing care and conscientiousness in one's work or duties",
        "partOfSpeech": "adjective",
        "synonyms": ["hardworking", "industrious"],
        "examples": ["She was a diligent student who never missed a deadline."],
        "antonyms": ["lazy", "idle"],
        "isFavoriteDefinition": false,
        "note": ""
      }
    ],
    "syllables": 3,
    "pronunciation": "'dɪlɪdʒənt",
    "isFavoriteWord": false
  },
  weitere Wörter
]
```

4.2 US2 – Definition favorisieren

4.2.1 Ablauf

1. Benutzer*in klickt Herz-Button bei Definition mit bestimmter id → PUT /api/words/{id}
2. Aktualisiertes Objekt, mit "isFavoriteDefinition: true", "isFavoriteWord: true" und einer möglichen Notiz wird abgeschickt
3. Bei Erfolg gibt das Backend das aktualisierte Word-Objekt zurück

4.2.2 Datentypen je Endpoint

PUT /api/words/4

Request Body:

```
{
  "word": "diligent",
  "results": [
    {
      "definition": "having or showing care and conscientiousness in one's work or duties",
      "partOfSpeech": "adjective",
      "synonyms": ["hardworking", "industrious"],
      "examples": ["She was a diligent student who never missed a deadline."],
      "antonyms": ["lazy", "idle"],
      "isFavoriteDefinition": true,
      "note": "Tolles Wort für Bewerbungen"
    }
  ],
  "syllables": 3,
  "pronunciation": "'dɪlɪdʒənt",
  "isFavoriteWord": true
}
```

Response-Status: 200 Ok

Response-Body: Gleich wie Objekt nur mit ID

Fehlerfall PUT /api/words/999 – ungültige ID:

Request-Body: Gleich wie oben

Response-Status: 404 Not Found

Response-Body:

```
{
  "timestamp": "2026-07-04T12:26:37.1648072",
  "status": 404,
  "message": "There's no word with id 999!"
}
```

4.3 US3 – Neues Wort mit mehreren Definitionen anlegen

4.3.1 Ablauf

1. Admin füllt Formular aus und schickt es ab
2. POST /api/words mit dem neuen Wort im Request Body
3. Frontend bekommt Response mit dem gleichen Inhalt + neu generierter ID und ändert seine Anzeige

4.3.2 Datentypen je Endpoint

POST /api/words

Request-Body:

```
{
  "word": "diligent",
  "results": [
    {
      "definition": "having or showing care and conscientiousness in one's work or duties",
      "partOfSpeech": "adjective",
      "synonyms": ["hardworking", "industrious"],
      "examples": ["She was a diligent student who never missed a deadline."],
      "antonyms": ["lazy", "idle"],
      "isFavoriteDefinition": false,
      "note": ""
    }
  ],
  "syllables": 3,
  "pronunciation": "'dɪlɪdʒənt",
  "isFavoriteWord": false
}
```

Response-Status: 201 Created

Response-Body: Gleich wie Request-Body nur mit generierter ID

Fehlerfall POST /api/words – fehlendes Pflichtfeld

Request-Body: Gleich wie oben, aber das "word"-Feld ist leer

Response-Status: 400 Bad Request

Response-Body:

```
{
  "timestamp": "2026-07-04T12:41:50.0941579",
  "status": 400,
  "message": "Validation failed",
  "fieldErrors": {
    "word": "word must not be blank"
  }
}
```

4.4 US4 – Wort löschen

4.4.1 Ablauf

1. Admin klickt auf Delete-Button
2. DELETE /api/words/{id} mit der richtigen ID
3. Bei Erfolg wird das Wort direkt aus der Datenbank gelöscht und das Frontend ändert seine Anzeige

4.4.2 Datentypen je Endpoint

DELETE /api/words/{id}

Request-Body: kein Body

Response-Status: 204 No Content

Response-Body: kein Body

Fehlerfall DELETE /api/words/999 – ungültige ID:

Request-Body: wie oben

Response-Status: 404 Not found

Response-Body:

```
{
  "timestamp": "2026-07-04T12:26:37.1648072",
  "status": 404,
  "message": "There's no word with id 999!"
}
```

5. Testplan

ID	Name	Vorbedingung*	Schritte**	Soll-Ergebnis	Durchführung	Status	Ist-Ergebnis	Bemerkung
T1	Wort per ID abrufen	Wort mit zu testender ID (z.B. 1) existiert	1. GET /api/words/1 in Insomnia eingeben	Response: 200 OK, vollständiges Word-Objekt mit allen Feldern	Am 11.07.2026 durch FO	OK	200 OK; vollständiges Wort-Objekt	-
T2	Neues Wort ohne Pflichtfeld anlegen		1. POST /api/words in Insomnia eingeben mit einem Request-Body, bei dem das "word"-Feld leer gelassen wurde.	Response: 400 Bad Request, Validierungsfehler	Am 11.07.2026 durch FO	OK	400 Bad Request; "Validation failed"; "word": "must not be blank."	-
T3	Wort löschen	- Wort mit zu testender ID (z.B. 1) existiert - T1 bestanden	1. DELETE /api/words/1 in Insomnia eingeben 2. GET /api/words/1 zum Testen, ob richtig gelöscht	Response DELETE: 204 No Content Response GET: 404 Not Found	Am 11.07.2026 durch FO	OK	DELETE: 204 No Content GET: 404 Not Found; "There's no word with id 1"	-

T4	Favorit setzen	Wort mit zu testender ID (z.B. 1) existiert	1. PUT /api/words/1 mit RequestBody mit Felder "isFavoriteDefinition: true" und "isFavoriteWord: true" in Insomnia eingeben	Response: 200 OK, gleiches Objekt wie im Request Body hier im Response Body, nur mit passender ID zusätzlich	Am 11.07.2026 durch FO	OK	200 OK; Änderungen sind im Response Body enthalten; ID ist gleichgeblieben;	-
T5	Wörter abrufen bei leerer Datenbank	Wörterbank hier leer	1. Datenbank im DBBeaver leeren mit TRUNCATE TABLE definition_synonyms, definition_examples, definition_antonyms, definitions, words RESTART IDENTITY CASCADE; 2. GET /api/words in Insomnia abrufen	Response: 200 OK, leerer Array im Response Body	Am 11.07.2026 durch FO	OK	200 OK; Response Body mit leerem Array: []	-

* Für jeden Testfall gilt, dass als Vorbedingung die App, Docker Desktop und der Container einwandfrei laufen, bzw. als erster Schritt gestartet werden. Ausserdem wird, ausser anders erwähnt, davon ausgegangen, dass min. 1 Wort schon in der DB existiert.

** Diese Schritte werden immer in Insomnia ausgeführt, ausser es wird anders beschrieben.

6. Installationsanleitung

6.1 Voraussetzungen

- Docker Desktop installiert und gestartet
- Git installiert
- Insomnia oder ähnliches Tool zum Testen (optional)
- DBeaver oder ähnliches Tool zum Anzeigen der Daten (optional)
- IntelliJ oder sonstiger IDE

6.2 Schritte

1. Repository in den gewünschten Ordner klonen
 - a. `git clone https://github.com/florinaosmani/wordbankbackend`
2. In den Projektordner wechseln
 - a. `cd wordbankbackend/backend`
3. Docker Compose starten. Dies startet automatisch die PostgreSQL Datenbank
 - a. `docker compose up -d`
4. App starten in IntelliJ oder sonstigem IDE
 - a. Auf den grünen Pfeil oben klicken
5. Die API ist nun erreichbar unter `http://localhost:8080/api/words`

Optional Anbindung an DBeaver:

1. DBeaver öffnen und oben links auf "Neue Verbindung" klicken
2. PostgreSQL auswählen und auf "Weiter" klicken
3. Folgende Verbindungsdaten eingeben
 - a. Host: localhost
 - b. Port: 5432
 - c. Database: wordbankdb
 - d. Username: user
 - e. Password: password
4. Auf "Verbindung testen" klicken um zu prüfen ob die Verbindung funktioniert
5. Auf "Fertigstellen" klicken
6. Die DB ist nun in DBeaver sichtbar

7. Hilfestellungen

Claude:

- Projektdokumentation
 - Unterstützung bei der Korrektur und Ideengenerierung
 - Klassendiagramm
 - Ist es Enumeration bei PartOfSpeech oder nicht?
- Projekt
 - Verbesserung der Formulierung der JavaDoc Kommentare
 - JavaDoc Erklärung
 - Wieso gibt es toDTO und toEntity gleichzeitig?
 - WordFormDTO, WordResponseDTO und das gleiche bei Definition
 - Erklärung wieso ich das brauche
 - DataSeeder
 - Wenn Seeding mit Json-Datei, wie verknüpfe ich Wort und Definition?
 - Repository
 - Advanced Query, aber für die Definition, also dem Child Entity
 - DefinitionFormDTO
 - Kompakter Konstruktor. Wie bekomme ich leere note als "" und nicht als null.
 - boolean kann kein NotBlank bekommen anscheinend.
 - WordFormDTO
 - Boolean und int haben kein NotBlank, sind automatisch false oder 0 wenn sie ausgelassen werden.
 - @Valid bei results, sonst werden die Definitionen nicht validiert.
 - GlobalExceptionHandler
 - Erklärung
 - JavaDoc
 - Verbesserung, Fehlerfindung, Erklärung.
 - Allgemein Bugs finden.